

Users's Manual for CPU Simulators

Brent Seidel
Phoenix, AZ

September 2, 2026

This document is ©2026 Brent Seidel. All rights reserved.

Note that this is a draft version and not the final version for publication.

Contents

1	Introduction	5
1.1	Disclaimer	5
1.2	License	5
2	How To Obtain	6
2.1	Dependencies	6
2.1.1	Ada Libraries	6
2.1.2	Other Libraries	7
3	Usage Instructions	8
3.1	Using Alire	8
3.2	Using gprbuild	8
4	API Description	9
4.1	General	9
4.2	Package Hierarchy	9
4.3	Numerical Data Types	10
4.4	Enumeration Data Types	11
5	The CPU Objects	13
5.1	Tracing	13
5.2	Primary API	14
5.3	Other API	15
5.3.1	Interrupt Related Routines	15
5.3.2	Halt/Running Related Routines	15
5.3.3	Interrupt Related Routines	16
5.3.4	Memory Related Routines	16
5.3.5	Memory Related Routines	17
5.3.6	Simulator Data Related Routines	17
5.3.7	Register Related Routines	18
5.4	Special Purpose API	19
6	The Bus Objects	21

7	The I/O Objects	27
7.0.1	Loading Data to Memory	28
7.0.2	Running a Simulation	29
7.0.3	Variants	29
7.0.4	Other	30
7.1	Example	30
7.2	8080 Family	30
7.3	68000 Family	30
7.4	6502 Family	31
7.5	PDP-11 Family	31
8	I/O Devices	32
8.1	Clock	33
8.2	Serial Ports	33
8.2.1	Basic Serial Port	33
8.2.2	Single Line Telnet Port	33
8.2.3	Multi-Line Telnet Port	33
8.2.4	Tape	33
8.3	Disk Interfaces	33
8.4	Device Names	34
	Bibleography	35

Chapter 1

Introduction

This project includes simulators for some existing processors. It can be embedded into other code as a library, or used stand-alone with a command line interpreter. The intent of these simulators are to provide instruction level simulation and not hardware level. Generally, no attempt has been made to count clock cycles - instructions may not even take the same relative amount of time to execute.

Interfaces are provided to allow the simulator to be controlled by and display data on a simulated control panel (see the Pi-Mainframe (<https://github.com/BrentSeidel/Pi-Mainframe>) project. These may be stubbed out or ignored if not needed.

1.1 Disclaimer

I have tried to make the documentation and software as accurate as possible. However, it is inevitable that errors exist. Should you find an error, feel free to write an issue on the GitHub project and I will look at it. For software errors especially, please provide information as to what change is needed. If you would like some particular hardware added, feel free to request that as well. Note that this is a hobby project maintained by one person who has other demands on their time.

1.2 License

This project is licensed using the GNU General Public License V3.0. Should you wish other licensing terms, contact the author.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Chapter 2

How To Obtain

This package is currently available on GitHub at <https://github.com/BrentSeidel/Sim-CPU>

2.1 Dependencies

2.1.1 Ada Libraries

The following Ada packages are used:

- `Ada.Calendar`
- `Ada.Characters.Latin_1`
- `Ada.Containers.Indefinite_Ordered_Maps`
- `Ada.Direct_IO`
- `Ada.Exceptions`
- `Ada.Integer_Text_IO`
- `Ada.Streams`
- `Ada.Strings.Fixed`
- `Ada.Strings.Maps.Constants`
- `Ada.Strings.Unbounded`
- `Ada.Text_IO`
- `Ada.Text_IO.Unbounded_IO`
- `Ada.Unchecked_Conversion`

2.1.2 Other Libraries

This library depends on the root package BBS available at <https://github.com/BrentSeidel/BBS-Ada> or using alire via “`alr get bbs`” and the BBS.Lisp package available at <https://github.com/BrentSeidel/Ada-Lisp> or using alire via “`alr get bbs_lisp`”.

Chapter 3

Usage Instructions

3.1 Using Alire

Alire automatically handles dependencies. To use this in your project, just issue the command “**alr with bbs_simcpu**” in your project directory. To build the standalone CLI program, first obtain the cli using “**alr get simcpucli**”. Change to the appropriate directory and use “**alr build**” and “**alr run**”. To use the load CP/M program, use “**alr get loadcpm**”. Change to the appropriate directory and use “**alr build**” and “**alr run**”.

3.2 Using gprbuild

This is a library of routines intended to be used by some program. To use these in your program, edit your *.gpr file to include a line to **with** the path to **bbs_simcpu_noalr.gpr**. Then in your Ada code **with** in the package(s) you need and use the routines.

This is also available as a standalone program with a command line interface (CLI). This can be used for development or debugging. To build the CLI, just build the **simcli_noalr.gpr** project file, after making sure that the dependencies are in the expected places.

There is another program **loadcpm** available to create a bootable CP/M floppy disk image. It is build using the **loadcpm_noalr.gpr** project. It takes an Intel hex file image and writes it to a floppy disk image.

Chapter 4

API Description

4.1 General

Several simulators are available for use. Each simulator may also have variation. So, one simulator may provide variations for different processors in a family of processors.

Each simulator is based on an object that derives from the `simulator` object defined in the `BBS.Sim_CPU` package. A generic simulator interface is defined with some procedures or functions that must be defined by a specific simulator and some that may be defined, if needed. There are also a number of utility functions that are not expected to be overridden by a specific simulator.

The current design has memory included in the simulator instead of being an external device. Any I/O decoding is also handled inside the simulator. This means that any reading and writing of memory has to be done via routines defined by the simulator.

4.2 Package Hierarchy

The packages for the simulator are arranged in a hierarchy to functionally decompose the simulation.

- `BBS.Sim_CPU` - This is the root package for the simulator.
 - `BBS.Sim_CPU.bus` - This is the root package for different kinds of data busses.
 - * `BBS.Sim_CPU.bus.mem8` - A simple 8 bit bus. Contains objects for a bus both with and without separate I/O space.
 - * `BBS.Sim_CPU.bus.pdp11` - A bus specific to the PDP-11 simulation. It could be a Unibus or Qbus.
 - `BBS.Sim_CPU.cpu` - This is the root package for the CPU simulations.
 - * `BBS.Sim_CPU.cpu.i8080` - This contains simulations for the Intel 8080, 8085, and the Zilog Z80. Further sub-packages are not listed.
 - * `BBS.Sim_CPU.cpu.m68000` - This contains simulations for the 68000 family of processors. Further sub-packages are not listed.
 - * `BBS.Sim_CPU.cpu.msc6502` - This contains the simulation for the 6502 processor.

- * `BBS.Sim_CPU.cpu.pdp11` - This contains simulations for the PDP-11 family of processors. Further sub-packages are not listed.
- `BBS.Sim_CPU.io` - This is the root package for all I/O devices.
 - * `BBS.Sim_CPU.io.bm792` - This is a PDP-11 boot ROM containing code to boot a RK05 disk on a RK11 controller.
 - * `BBS.Sim_CPU.io.clock` - This contains a generic clock device.
 - `BBS.Sim_CPU.io.clock.kw11` - This is a KW11 clock for PDP-11 computers
 - * `BBS.Sim_CPU.io.disk` - This contains definitions common for all disk type devices
 - `BBS.Sim_CPU.io.disk.floppy` - This contains generic floppy and hard disk controllers.
 - `BBS.Sim_CPU.io.disk.rk11` - This is a simulation of a RK11 controller for RK05 disks for PDP-11 computers.
 - `BBS.Sim_CPU.io.disk.rk611` - This is a simulation of a RK611 controller for RK07 disks (RK06 disks are not implemented) for PDP-11 computers.
 - `BBS.Sim_CPU.io.disk.tm11` - This is a simulation of a TM11 tape drive controller for PDP-11 computers. It may eventually move to the ...tape package.
 - * `BBS.Sim_CPU.io.kt11` - This is a simulation of a KT11 memory management unit for PDP-11 computers.
 - * `BBS.Sim_CPU.io.serial` - This contains definitions and a basic implementation for serial interfaces.
 - `BBS.Sim_CPU.io.serial.dl11` - This is a simulation of a DL11 single line interface for PDP-11 computers.
 - `BBS.Sim_CPU.io.serial.dz11` - This is a simulation of a DZ 8 line terminal multiplexer for PDP-11 computers.
 - `BBS.Sim_CPU.io.serial.mux` - This is a basic 8 line terminal multiplexer.
 - `BBS.Sim_CPU.io.serial.telnet` - This is a basic single line interface.
 - * `BBS.Sim_CPU.io.tape` - Contains definitions and an implementation of a basic paper tape device.
 - `BBS.Sim_CPU.io.tape.pc11` - This is a simulation of a PC11 paper tape reader/punch for PDP-11 computers.

4.3 Numerical Data Types

Currently, processors with address and data busses up to 32 bits wide are supported. For the address bus, use the data type `addr_bus` and for the data bus, use `data_bus`. These are both defined as a 32 bit unsigned integer. Each simulator may use as many or as few of these bits as are needed. This means that if (as in most cases) a simulator doesn't define a full 4 GB of memory, the external program can try to read or write non-existent memory. The bus object should check for this, otherwise this will probably cause an exception.

4.4 Enumeration Data Types

The `proc_mode` enumeration is used to indicate the processor mode. Not all modes will be used by any given processor and the meanings are processor specific. Any of these modes could be used by a processor that doesn't define multiple modes. The defined modes are:

- *PROC_NONE* - Unspecified processor mode.
- *PROC_KERN* - Kernel mode. This is usually the most privileged mode.
- *PROC_EXEC* - Executive mode. This is a little less privileged.
- *PROC_SUP* - Supervisor mode. This is a little less privileged.
- *PROC_USER* - User mode. This is usually the least privileged mode.

The `addr_type` enumeration is used to indicate the type of memory access. They may be used for controlling access to memory. The defined types are:

- *ADDR_NONE* - Unspecified address type.
- *ADDR_INTR* - Reading interrupt data, typically an interrupt vector.
- *ADDR_DATA* - Reading or writing data.
- *ADDR_INS* - Reading instructions (probably not writing).
- *ADDR_IO* - Reading or writing to I/O ports for processors that have separate I/O addresses.

The `bus_type` enumeration defines the type of bus for attaching I/O devices and bus accesses. This can either be the I/O bus (for processors with separate I/O space) or the memory bus for memory and memory mapped I/O. The defined busses are:

- *BUS_MEMORY* - Use the memory bus.
- *BUS_IO* - Use the I/O bus

The `bus_size` enumeration defines the desired size of a bus transaction. The defined sizes are:

- *bits0* - Zero bit (null) transaction. Not currently used.
- *bits8* - Eight bit (byte) transaction.
- *bits16* - Sixteen bit (word) transaction.
- *bits32* - Thirty-two bit (long-word) transaction. Not currently used.

The `bus_stat` enumeration is used for reporting the result of a bus transaction. The defined results are:

- *BUS_SUCC* - The transaction succeeded. This is the result that you hope you get.
- *BUS_NONE* - Nothing exists at this address. Either uninstalled memory or non-existent I/O device.

- *BUS_ALIGN* - Illegal alignment, such as an odd address
- *BUS_PROT* - A protection violation occurred. This is typically reported by a memory management unit when the `proc_mode` doesn't match the required level for the address.
- *BUS_PARITY* - A parity error occurred during the bus transaction. This would usually be used for testing error handlers.
- *BUS_MMU* - Error was reported by memory management unit
- *BUS_ECC1* - A correctable ECC error occurred during the bus transaction. This would usually be used for testing error handlers.
- *BUS_ECC2* - An uncorrectable ECC error occurred during the bus transaction. This would usually be used for testing error handlers

Chapter 5

The CPU Objects

The CPU object is responsible for executing instructions. Generally before using a simulator, it must be initialized, a variant selected, and a bus attached. However this is all implementation dependent.

5.1 Tracing

Depending of flag settings, information can be displayed during simulator processing. The API presents this as a `trace_flags` record value. Two `Unchecked_Conversion` functions are provided for converting between this and `uint32`. These routines are `uint32_to_trace` and `trace_to_uint32`. Currently only the 6 LSBs are used, the rest are reserved for future use.

- 0 List instructions being traced.
- 1 List I/O operations.
- 2 List data operations.
- 3 List bus specific items.
- 4 List control transfers.
- 5 List exceptions.

Set the trace flags.

```
procedure trace(self : in out simulator; l : trace_flags);
```

- `self` - The simulator object.
- `l` - The trace flags

5.2 Primary API

To initialize the CPU, use:

```
procedure init(self : in out simulator) is abstract;
```

- **self** - The simulator object to initialize.

To select a CPU variant, use:

```
procedure variant(self : in out simulator; v : Natural) is abstract;
```

- **self** - The simulator object to initialize.
- v - The index of the variant to use. This value is defined by the specific CPU object.

Attaching a bus to a CPU also requires attaching the cpu to the bus. This gives both the CPU and the bus object a reference to each other. The routine to attach a bus to a CPU automatically calls the routine to attach the CPU the bus. To attach a bus, use (note that this is declared as null since some CPUs may not need this and thus they won't have to create a null implementation):

```
procedure attach_bus(self : in out simulator;  
                    bus : BBS.Sim_CPU.bus.bus_access;  
                    index : Natural) is Null;
```

- **self** - The simulator object to initialize.
- **bus** - The bus object to attach.
- **index** - The CPU index. Expected to be used for future multi-CPU simulations. Currently unused by all defined bus objects.

Called once when Start/Stop switch is moved to start position. This typically sets the program counter, but may do other CPU specific things.

```
procedure start(self : in out simulator) is Null;
```

- **self** - The simulator object.

Called to start simulator execution at a specific address. This is made null rather than abstract so that simulators that don't use it don't need to override it.

```
procedure start(self : in out simulator; addr : addr_bus) is null;
```

- **self** - The simulator object.

Called once per frame to execute the next instruction.

```
procedure run(self : in out simulator) is Null;
```

- **self** - The simulator object.

5.3 Other API

Some of these routines are only used in specific cases, such as when interfaced to switches and light. Others have more general application.

Clear a breakpoint at a specific address.

```
procedure clearBreak(self : in out simulator; addr : addr_bus) is null;
```

- **self** - The simulator object.
- **addr** - The address.

5.3.1 Interrupt Related Routines

Cancel an interrupt if not already taken.

```
procedure cancelInterrupt(self : in out simulator; data : long) is null;
```

- **self** - The simulator object.
- **data** - The interrupt code. Interpreted by the simulator.

5.3.2 Halt/Running Related Routines

Clear the halted flag to allowing processing to continue.

```
procedure continue_proc(self : in out simulator) is null;
```

- **self** - The simulator object.

Force a simulator to enter a halt state (if implemented). Can be used for some error conditions where the CPU needs to be halted.

```
procedure halt(self : in out simulator) is null;
```

- **self** - The simulator object.

Check if simulator is halted.

```
function halted(self : in out simulator) return Boolean is (False);
```

- **self** - The simulator object.

Request a processor reset.

```
procedure reset(self : in out simulator) is null;
```

- **self** - The simulator object.

Sets a breakpoint at a specific address.

```
procedure setBreak(self : in out simulator; addr : addr_bus) is null;
```

- **self** - The simulator object.
- **addr** - The address.

Return the current value of the trace flags.

```
function trace(self : in out simulator) return trace_flags;
```

- **self** - The simulator object.

5.3.3 Interrupt Related Routines

Signal an interrupt from a device.

```
procedure interrupt(self : in out simulator; data : long) is null;
```

- **self** - The simulator object.
- **data** - The interrupt code. Interpreted by the simulator.

Allow an external user to enable or disable processing of interrupts

```
procedure interrupts(self : in out simulator; state : Boolean) is null;
```

- **self** - The simulator object.
- **state** - *True* for enable, *False* for disable.

Returns simulator dependent status of interrupts as `int32`. This may be something like enabled/disabled or processor priority.

```
function intStatus(self : in out simulator) return int32 is (0);
```

- **self** - The simulator object.

5.3.4 Memory Related Routines

Note that memory could be accessed more directly via the bus object.

Load data into the simulator. The simulator is responsible for interpreting **name**, decoding, and loading the data.

```
procedure load(self : in out simulator; name : String) is null;
```

- **self** - The simulator object.
- **name** - Typically used as a file name.

5.3.5 Memory Related Routines

Read a value from an address in simulator memory. If simulator has memory mapped I/O, this could also be an I/O device.

```
function read_mem(self : in out simulator; mem_addr : addr_bus)
    return data_bus is (0);
```

- **self** - The simulator object.
- **mem_addr** - The memory address to access.

Write a value to an address in simulator memory. If simulator has memory mapped I/O, this could also be an I/O device.

```
procedure set_mem(self : in out simulator; mem_addr : addr_bus;
    data : data_bus) is Null;
```

- **self** - The simulator object.
- **mem_addr** - The memory address to access.
- **data** - The value to write to the memory address.

5.3.6 Simulator Data Related Routines

Returns the simulator name as a string.

```
function name(self : in out simulator) return String is ("No-simulator");
```

- **self** - The simulator object.

Return a simulator option value as a string. This is highly simulator specific.

```
function option(self : in out simulator;
    opt : String) return String is ("");
```

- **self** - The simulator object.
- **opt** - The name of the option.

Set a simulator specific option. This is highly simulator specific.

```
procedure option(self : in out simulator;
    opt : String; value : String) is null;
```

- **self** - The simulator object.
- **opt** - The name of the option.
- **value** - The new value for the option.

Returns the variant name as a string.

```
function variant(self : in out simulator;  
                v : Natural) return String is ("No-variant");
```

- **self** - The simulator object.

Returns the current variant index as a **Natural**.

```
function variant(self : in out simulator) return Natural is (0);
```

- **self** - The simulator object.

Set the variant to the specified value.

```
procedure variant(self : in out simulator; v : Natural) is Null;
```

- **self** - The simulator object.
- **v** - The variant index.

Returns the number of variants as a **Natural**.

```
function variants(self : in out simulator) return Natural is (1);
```

- **self** - The simulator object.

5.3.7 Register Related Routines

Return the value of the specified register as **uint32**.

```
function read_reg(self : in out simulator; num : uint32)  
              return data_bus is (0);
```

- **self** - The simulator object.
- **num** - The register.

Return the value of the specified register as a **String**. This would normally be used, for example, to return a formatted representation of the condition codes or PSW.

```
function read_reg(self : in out simulator; num : uint32)  
              return String is ("No-Reg");
```

- **self** - The simulator object.
- **num** - The register.

Returns the number of simulator registers as a **uint32**. What constitutes a register is simulator dependent.

function registers(self : in out simulator) **return** uint32 **is** (0);

- **self** - The simulator object.

Returns a **String** representing the name of the specified register.

function reg_name(self : in out simulator; num : uint32)
 return String **is** ("No-Reg");

- **self** - The simulator object.
- **num** - The register.

Sets the specified register to the specified value.

procedure set_reg(self : in out simulator; num : uint32;
 data : data_bus) **is** Null;

- **self** - The simulator object.
- **num** - The register.
- **data** - The value for the register.

5.4 Special Purpose API

While the routines in this section are publicly available, they are intended for specific applications.

Called once when the Deposit switch is moved to the Deposit position. Used for blinkenlights.

procedure deposit(self : in out simulator) **is** Null;

- **self** - The simulator object.

Called once when the Examine switch is moved to the Examine position. Used for blinkenlights.

procedure examine(self : in out simulator) **is** Null;

Return the last address used when writing on the I/O bus. Obviously, this only applies to CPUs that have separate I/O space.

function lastOutAddr(self : in out simulator) **return** addr_bus **is** (0);

- **self** - The simulator object.

Return the last data written when writing on the I/O bus. Obviously, this only applies to CPUs that have separate I/O space.

function lastOutData(self : in out simulator) **return** data_bus **is** (0);

- **self** - The simulator object.

Override an I/O read operation at a specified address with specified data. Obviously, this only applies to CPUs that have separate I/O space.

```
procedure overrideIn(self : in out simulator; addr : in addr_bus;  
                    data : in data_bus) is null;
```

- **self** - The simulator object.
 - **addr** - The address to override.
 - **data** - The value to provide for a read to the specified address.
-

Chapter 6

The Bus Objects

This bus object contains the memory for the CPU and interfaces with the I/O devices. Once the bus object is created, the main action a host program needs to do is to attach I/O devices.

```
procedure attach_io(self : in out bus;  
                    io_dev : BBS.Sim_CPU.io.io_access;  
                    base_addr : addr_bus;  
                    which_bus : bus_type) is abstract;
```

- **self** - The bus object to initialize.
- **io_dev** - The I/O device object to attach to the bus.
- **base_addr** - The base address to use to access the device registers.
- **which_bus** - One of the defined **bus_type** values.

The bus object provides routines for accessing memory by physical or logical addresses. In some cases these will be identical. When some sort of address translation is provided, such as bank switching or a full MMU, they may be different. All memory access routines provide a **status** parameter that contains the **bus_state** the status of the operation. If **status** is not *BUS_SUCC*, the operation did not complete and any data returned by a read operation should be ignored.

To read from physical memory, use:

```
function readp(self : in out bus; addr : addr_bus; status : out bus_stat)  
            return data_bus is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **status** - the **bus_status** result of the operation.
- Returns the value at the specified address.

To write to physical memory, use:

```
procedure writep(self : in out bus; addr : addr_bus; data: data_bus;  
                status : out bus_stat) is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **data** - The value to write to memory.
- **status** - the **bus_status** result of the operation.

To read from logical memory, use:

```
function readl(self : in out bus; addr : addr_bus; mode : proc_mode;  
               addr_kind : addr_type; status : out bus_stat)  
  return data_bus is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.
- Returns the value at the specified address.

To write to logical memory, use:

```
procedure writel(self : in out bus; addr : addr_bus; data: data_bus;  
                mode : proc_mode; addr_kind : addr_type;  
                status : out bus_stat) is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **data** - The value to write to memory.
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.

For reading from logical memory in bytes, words, and long words in LSB first or MSB first format, the following are provided:

LSB First:

```
function readl8l(self : in out bus; addr : addr_bus; mode : proc_mode;  
                addr_kind : addr_type; status : out bus_stat)  
    return byte is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.
- Returns the **byte** at the specified address.

```
function readl16l(self : in out bus; addr : addr_bus; mode : proc_mode;  
                  addr_kind : addr_type; status : out bus_stat)  
    return word is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.
- Returns the **word** at the specified address with LSB first.

```
function readl32l(self : in out bus; addr : addr_bus; mode : proc_mode;  
                  addr_kind : addr_type; status : out bus_stat)  
    return long is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.
- Returns the **long** at the specified address with LSB first.

MSB First:

```
function readl8m(self : in out bus; addr : addr_bus; mode : proc_mode;  
                addr_kind : addr_type; status : out bus_stat)  
    return byte is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.
- Returns the **byte** at the specified address (this is the same as readl8l).

```
function readl16m(self : in out bus; addr : addr_bus; mode : proc_mode;  
                 addr_kind : addr_type; status : out bus_stat)  
    return word is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.
- Returns the **word** at the specified address with MSB first.

```
function readl32m(self : in out bus; addr : addr_bus; mode : proc_mode;  
                 addr_kind : addr_type; status : out bus_stat)  
    return long is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.
- Returns the **long** at the specified address with MSB first.

For writing to logical memory in bytes, words, and long words in LSB first or MSB first format, the following are provided:

LSB first:

```
procedure writel8l(self : in out bus; addr : addr_bus;  
                  data: byte; mode : proc_mode;  
                  addr_kind : addr_type; status : out bus_stat) is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **data** - The byte to write to memory.
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.

```
procedure writel16l(self : in out bus; addr : addr_bus;  
                   data: byte; mode : proc_mode;  
                   addr_kind : addr_type; status : out bus_stat) is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **data** - The word to write to memory (LSB first).
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.

```
procedure writel32l(self : in out bus; addr : addr_bus;  
                   data: byte; mode : proc_mode;  
                   addr_kind : addr_type; status : out bus_stat) is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **data** - The long to write to memory (LSB first).
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.

MSB first:

```
procedure writel8m(self : in out bus; addr : addr_bus;  
                  data: byte; mode : proc_mode;  
                  addr_kind : addr_type; status : out bus_stat) is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **data** - The **byte** to write to memory.
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.

```
procedure writel16m(self : in out bus; addr : addr_bus;  
                   data: byte; mode : proc_mode;  
                   addr_kind : addr_type; status : out bus_stat) is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **data** - The **word** to write to memory (MSB first).
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.

```
procedure writel32m(self : in out bus; addr : addr_bus;  
                   data: byte; mode : proc_mode;  
                   addr_kind : addr_type; status : out bus_stat) is abstract;
```

- **self** - The bus object to access.
- **addr** - The memory address to read.
- **data** - The **long** to write to memory (MSB first).
- **mode** - The **proc_mode** for the operation.
- **addr_kind** - The **addr_type** for the operation.
- **status** - the **bus_status** result of the operation.

Chapter 7

The I/O Objects

I/O objects are used to simulate various I/O devices. From the viewpoint of a CPU, an I/O device consists of a contiguous range of addresses for the device registers on either the memory or I/O bus. These device registers are used to interface with and control the device. Once a device has been attached to a bus, the CPU may communicate with it via the device registers.

If a device uses interrupts, the owning CPU must be set along with an exception code. If a device uses DMA, the owning CPU must be specified (note that this may change to be the bus that the device is attached to).

To set the owning CPU, use:

```
procedure setOwner(self : in out io_device;  
                  owner : BBS.Sim_CPU.CPU.sim_access);
```

- **self** - The I/O object to access.
- **owner** - The CPU object for the owning CPU.

To set the exception code, use:

```
procedure setException(self : in out io_device;  
                      except : long) is abstract;
```

- **self** - The I/O object to access.
- **except** - The exception code to use. The interpretation of this code depends on the specific CPU.

To write to the device registers use:

```
procedure write(self : in out io_device; addr : addr_bus;  
               data : data_bus) is abstract;
```

- **self** - The I/O object to access.
- **addr** - The address of the device register to access.

- **data** - The data to write to the device register.

To read from the device registers use:

```
function read(self : in out io_device; addr : addr_bus)
    return data_bus is abstract;
```

- **self** - The I/O object to access.
- **addr** - The address of the device register to access.
- Returns the value from the specified device register.

7.0.1 Loading Data to Memory

The main routines for reading and writing simulator memory are:

Called to set a memory value.

```
procedure set_mem(self : in out simulator;
    mem_addr : addr_bus;
    data : data_bus) is abstract;
```

- **self** - The simulator owning the memory to set.
- **mem_addr** - The memory address. This is allowed to be a 32 bit value. The simulator itself may do range limiting or wrapping (eg. only low order 16 bits are used) internally.
- **data** - The data value to write into the memory address. This is allowed to be a 32 bit value. The simulator decides how to handle this value and what value actually gets written.

Called to read a memory value.

```
function read_mem(self : in out simulator;
    mem_addr : addr_bus) return
    data_bus is abstract;
```

- **self** - The simulator owning the memory to read.
- **mem_addr** - The memory address. This is allowed to be a 32 bit value. The simulator itself may do range limiting or wrapping (eg. only low order 16 bits are used) internally.
- Returns the value of memory at the specified memory address.

The actual addressing and data bus used are defined by the specific simulator.

For loading bulk data into memory use:

```
procedure load(self : in out simulator; name : String)
    is null;
```

Its implementation is defined by the simulator. Typically it loads an Intel Hex file or Motorola S-Record file representing a memory image.

7.0.2 Running a Simulation

The **start** procedure is called first to specify a starting address for program execution. This is called to start simulator execution at a specific address. This is made null rather than abstract so that simulators that don't use it don't need to override it.

```
procedure start(self : in out simulator; addr : addr_bus) is null;
```

Then, each instruction is individually executed using the **run** procedure. This is called once per frame when start/stop is in the start position and run/pause is in the run position.

```
procedure run(self : in out simulator) is abstract;
```

Certain conditions or instruction can cause a simulator to halt. The **halted** and **continue_proc** routines can be used to test for this condition and clear it.

Check if simulator is halted. If a simulator doesn't define this, it will return *False*.

```
function halted(self : in out simulator) return Boolean is (False);
```

This clears the halted flag allowing processing to continue. If a simulator doesn't define this, it has no action.

```
procedure continue_proc(self : in out simulator) is null;
```

From all of this, one can write the core of a simulator as follows:

```
—
— A bunch of other stuff including defining some
— simulator as "sim"
—
begin
    sim.init;
    —
    — I/O devices can be added here...
    —
    sim.load("image.hex");
    sim.start(0); — Start execution at address 0
    while not sim.halted loop
        sim.run;
    end loop;
end
```

7.0.3 Variants

Each simulator can support variants. This enables one simulator to support multiple CPUs in a family. Since the variants supported are unique to each simulator a universal data type cannot be used. Variants are identified by a **Natural** number. The following routines are used to get the number of variants supported by a simulator, the name of each variant, the currently selected variant, and to select a variant:

Called to get number of variants. This defaults to returning one if the simulator doesn't define an alternative.

```
function variants(self : in out simulator) return Natural is (1);
```

Called to get variant name. This needs to be overridden by the simulator.

```
function variant(self : in out simulator; v : Natural)
  return String is abstract;
```

Called to get current variant index. This needs to be overridden by the simulator.

```
function variant(self : in out simulator) return Natural is abstract;
```

Called to set variant. This needs to be overridden by the simulator.

```
procedure variant(self : in out simulator; v : Natural) is abstract;
```

7.0.4 Other

A number of other functions are defined to support I/O devices, interfaces with a front panel, and other things. The I/O support routines will be discussed more in chapter 8 on I/O devices.

7.1 Example

The example simulator provides an example of using the simulator object interface. Its primary purpose is to blink the lights in interesting ways in the Pi-Mainframe (<https://github.com/BrentSeidel/Pi-Mainframe>) project. There are a number of different patterns selectable. Variants are defined for “Copy Switches”, “Count”, “16-Bit Scan”, “16-Bit Bouncer”, “Fibonacci Counter”, “32-Bit Scan”, and “32-Bit Bouncer”.

This simulator is unusual in that it has no memory defined, but instead has several registers defined that act as memory. When reading or writing memory, the address is ignored and the value returned depends on the pattern (variant) selected.

7.2 8080 Family

The 8080 simulator has variants defined for the 8080, 8085, and Z-80 processors. These are 8 bit processors with an 8 bit data bus and a 16 bit address bus. In addition to a memory bus, these processors also include an I/O bus with 8 bit I/O port addressing. See [12] for more information about the 8080/8085. See [17] and [16] for more information about the Z-80.

Currently, the 8080 family does not have memory mapped I/O. This may be added at some time in the future.

When loading data using the `load` routine, the specified file is assumed to be in Intel Hex format.

7.3 68000 Family

The 68000 simulator has variants defined for the 68000, 68008, 68010, and CPU32. Only the 68000 and 68008 are currently implemented. The 68010 and CPU32 are for future development. Internally, these are 32 bit processors with 32 bit data and 32 bit address busses. These processors do not have a separate I/O address space. All I/O is mapped as part of the main memory. The

external address and data bus sizes depend on the variant selected. See [13] and [14] for more information about the 68000 family.

The interrupt code is interpreted with the low order bits (7-0) representing the vector number and the next 8 bits (15-8) representing the priority. Interrupts with vectors that match internally defined exception vectors are ignored. Thus only interrupt vectors in the range 25-31 and 64-255 are processed.

When loading data using the `load` routine, the specified file is assumed to be in Motorola S-Record format.

7.4 6502 Family

The 6502 simulator currently has only a working variants defined for the 6502 processor. This is an 8 bit processors with an 8 bit data bus and a 16 bit address bus. This processor does not have a separate I/O address space. All I/O is mapped as part of the main memory. Three interrupts are define for normal interrupts, non-maskable interrupts, and reset. See [15] for more information about the 6502.

When loading data using the `load` routine, the specified file is assumed to be in Intel Hex format.

7.5 PDP-11 Family

Most of the 16 bit addressing members are supported, except the LSI-11, PDP-11/03. Future models with memory management are planned at some point. This is a line of 16 bit minicomputers that was developed in the 1970s and was popular through the 1990s. Models with no memory management had 16 address bit. With memory management, depending on the model, addresses of 18 or 22 bits was supported. These processors do not have a separate I/O address space. All I/O is mapped as part of the main memory. By convention, all I/O is located in the upper 8kBytes of memory. See [10] and [11] for more information about the PDP-11 in general.

A wealth of information about the PDP-11 (and other DEC hardware) is on bitsavers at <https://www.bitsavers.org/pdf/dec/>. Software can also be found on bitsavers at <https://www.bitsavers.org/bits/DEC/>. Various diagnostic software can be found using the PDP-11 Diagnostics - Database at <https://www.retrocmp.com/tools/pdp-11-diagnostic-database/202-pdp-11-diagnostics-database>.

Chapter 8

I/O Devices

Each I/O device is based on an object that derives from the `io_device` object defined in the `BBS.Sim_CPU` package. A generic I/O device interface is defined with some procedures or functions that must be defined by a specific I/O device and some that may be defined, if needed. There are also a number of utility functions that are not expected to be overridden by a specific I/O device.

To be used by a simulator, an I/O device must first be attached to the simulator using the `attach_io` routine. This is called to attach an I/O device to a simulator at a specific address. Bus is simulator dependent as some CPUs have separate I/O and memory space.

```
procedure attach_io(self : in out simulator;  
                    io_dev : io_access;  
                    base_addr : addr_bus;  
                    bus : bus_type) is abstract;
```

Many of the I/O devices will have an initialization routine specific for that device. Since this may have device specific parameters, it can't be defined as part of the `io_device` object.

Along with attaching the I/O device to the simulator, the base address needs to be set in the I/O device object. The value for `base` in this routine should match the value for `base_addr` in the `attach_io` routine. Bad things may happen if they don't match.

```
procedure setBase(self : in out io_device;  
                  base : addr_bus) is abstract;
```

The I/O device `read` and `write` functions are called by the simulator when an address within the devices address range is accessed. Note that the address range starts at the `base_addr` specified in the `attach_io` call and extends through the number of addresses specified in the `getSize` function. If needed, the simulator can query the I/O device for its base address using the `getBase` function. If a device needs to do DMA or interrupt a simulator, the `setOwner` procedure must be called first to give the device a reference to the simulator. For interrupts, the `setException` routine is used to define a `long` value that the device passes to the simulator on exception.

```
procedure setException(self : in out io_device;  
                       except : long);
```

If a device does not use interrupts or DMA, this routine can be declared as `null`.

8.1 Clock

The main function of the clock device is to provide a periodic interrupt. It can be enabled or disabled and the time between interrupts can be set in $\frac{1}{10}$ th of a second. The base time can be changed by editing the code. Note that the timing is only approximate and depends on many things. Also since the simulation is not running at the same speed as actual hardware the number of instructions between each interrupt will be different from actual hardware.

8.2 Serial Ports

Serial ports provide a way to provide a terminal interface to a simulator. Unlike real serial ports, output buffering is done by the host operating system and it is not obvious to the I/O device if the output has been completed or not. As a result, currently no status is set or interrupts generated on output. The output is assumed to complete instantly.

8.2.1 Basic Serial Port

The basic serial port was developed in early testing to send output to and get input from the terminal controlling the simulator. It worked well enough to test some initial concepts, but has been superseded by the single line telnet port.

8.2.2 Single Line Telnet Port

The single line telnet port provides a replacement for the basic serial port. This device has an `init` routine that is used to set the TCP port. Using `telnet` to connect to this port connects to the device and will allow communication with software running on the simulator. If enabled, this device can provide interrupts to the simulator when characters are received.

8.2.3 Multi-Line Telnet Port

The multi-line telnet port combines 8 ports into a single simulated device. The `init` routine specifies the starting TCP port. It and the next seven ports can be accessed by `telnet` to communicate with software running on the simulator. This uses a single exception and requires fewer addresses than having 8 single line telnet ports.

8.2.4 Tape

The “paper” tape interface is based on a serial port. From the point of view of the host program, it reads bytes and writes bytes. The read bytes come from an attached file and the written bytes are written to an attached file (note that there should be two different files). One could also imagine this as a cassette tape interface if that better fits the computer you’re simulating.

8.3 Disk Interfaces

The initial `disk_ctrl` device was designed with 8 inch floppy disks in mind for use by CP/M. It has been extended to allow other disk geometries, but follows the model of tracks, sectors per track,

and heads for defining a disk. It currently supports 16 bit DMA, but may be extended to support 32 bit DMA. All data transfers occur between the request and the next instruction processed by the simulator.

Additional disk controllers based on actual hardware have been developed to support the PDP-11 CPU. Enough of the hardware has been simulated to support the RT-11 operating system. The controllers supported here are RK11 and RK611. The TM11 tape controller is also internally treated as a disk drive. This may change in the future.

8.4 Device Names

Each device has a name that can be used to refer to it. The name consists of an alphabetic device code followed by a decimal unit number. If no unit number is specified, 0 is assumed. A unit number of zero is special. It is used to refer to a generic device, such as when attaching a device - the unit number is assigned during the attachment. The currently defined device codes are (note that not all are implemented in the CLI, but will eventually be):

CLK – A clock device to generate a periodic interrupt.

CON – A console serial device. In most cases, this should not be used.

FD – Main disk controller with a track and sector interface. It started out as a floppy disk controller, but geometries can be defined beyond that.

HD – An experimental disk controller using just block numbers. It is not yet implemented and tested.

MUX – An 8 channel terminal multiplexer using a telnet interface.

PRN – A simple printer output device.

PTP – A paper tape interface. It can read and write simulated tapes.

TEL – A single channel terminal device using a telnet interface.

The following devices simulate actual hardware for PDP-11 computers.

DL11 – A single line serial interface, see [6].

DZ11 – An 8 line serial interface multiplexer, see [9].

KW11 – A clock for the PDP-11 series of computer, see [1].

RK11 – A disk controller that can have up to 8 attached RK05 simulated drives, see [5].

RK611 – A disk controller than can have up to 8 attached RK07 simulated drives (RK06 drives are not supported), see [8].

PC11 – A combination paper tape reader/punch, see [3].

TM11 – A magnetic tape controller that can have up to 8 attached tape drives. Note that this is internally treated as a disk drive so use the disk command to control it, see [7].

Bibliography

- [1] Digital. *KW11-L Line Time Clock Manual*. Digital Equipment Corporation, 1973.
- [2] Digital. *PDP-11/05 Computer Manual*. Digital Equipment Corporation, 1973.
- [3] Digital. *PC11 High-Speed Reader/Punch and Control Manual*. Digital Equipment Corporation, 1974.
- [4] Digital. *PDP11 04/05/10/35/40/45 Processor Handbook*. Digital Equipment Corporation, 1975.
- [5] Digital. *RK11-D and RK11-E Moving Head Disk Drive Controller Manual*. Digital Equipment Corporation, 1975.
- [6] Digital. *DL11 Asynchronous Line Interface User's Manual*. Digital Equipment Corporation, 1976.
- [7] Digital. *TMB11/TS03 DECmagtape System User's Manual (TMB11-M System)*. Digital Equipment Corporation, 1976.
- [8] Digital. *RK06/RK07 Disk Drive User's Manual*. Digital Equipment Corporation, 1978.
- [9] Digital. *DZ11 User's Guide*. Digital Equipment Corporation, 1979.
- [10] Digital. *PDP11 04/34a/44/60/70 Processor Handbook*. Digital Equipment Corporation, 1981.
- [11] Digital. *PDP-11 Architecture Handbook*. Digital Equipment Corporation, 1984.
- [12] Intel. *8080/8085 Assembly Language Programming*. Intel, 1978.
- [13] Motorola. *M68000 16/32-Bit Microprocessor Programmer's Reference Manual*. Prentice-Hall, 4th edition, 1984.
- [14] Motorola/NXP. *Motorola M68000 Programmer's Reference Manual*. Motorola/NXP, 1992.
- [15] MOS Technology. *MSC6502 Microcomputer Family Programming Manual*. MOS Technology, July 1976.
- [16] Sean Young and Jan Wilmans. *The undocumented z80 documented*, 2005.
- [17] Zilog/IXYS. *Z80 CPU User Manual*. Zilog, 2016.